Sistema de diálogos

Guía de implementación y uso | Revisión: 6 de noviembre de 2026

Estado: implementado; la verificación visual y de interacción debe hacerse en Unity.

Fecha de revisión: 6 de noviembre de 2026.

Esta guía explica el sistema de conversaciones que usan los NPC. La implementación se encuentra principalmente en `Assets/_SocialProyectoCenigraf/Scripts/Dialogue/` y `Assets/_SocialProyectoCenigraf/Scripts/World/NPC/`.

¿Qué es?

El sistema de diálogos permite crear conversaciones para los personajes del juego sin programar cada frase en el comportamiento del NPC. El contenido se guarda en un `DialogueCatalog` (un asset de Unity) y el NPC guarda una referencia a ese catálogo junto con el ID de la conversación que debe iniciar.

Una conversación puede ser lineal o estar compuesta por nodos conectados:

- Una **conversación lineal** usa una lista de líneas que se presentan en orden. Se conserva para compatibilidad con diálogos antiguos.
- Una **conversación por nodos** guarda todas sus líneas y decisiones en una sola entrada del catálogo. Cada nodo tiene un ID, interlocutor, texto y destinos. Las respuestas llevan a otros nodos o terminan la conversación. Las ramas no necesitan convertirse en entradas separadas del catálogo.

El mismo catálogo puede ser presentado por dos vistas:

- **DialogoOfficial:** ventana emergente con tipografía estándar, retrato del hablante y opciones seleccionables. Es la presentación normal para los diálogos por nodos.
- **DialoguePresenter:** presentación cinematográfica pixelada, con texto que aparece progresivamente y opciones de navegación vertical. Mantiene los diálogos lineales existentes y también puede presentar nodos cuando el modo Debug de la escena lo solicita.

Las clases principales son `DialogueCatalog`, `DialogueSequence`, `DialogueNode`, `DialogueChoice`, `DialoguePresenter` y `DialogoOfficial`. Para conectar un NPC se usa `NPCDialogoOfficialInteraction` o el controlador de diálogo anterior que ya tenga la escena.

¿Para qué se usa?

Se usa para:

- Presentar información de personajes y del mundo.
- Dejar que el jugador escoja qué tema consultar y conectar cada respuesta con una rama diferente.
- Cambiar el inicio o el avance de una conversación según el inventario, el historial de conversaciones o el progreso de misiones.
- Personalizar el texto con el nombre del jugador.
- Al completar una conversación o una rama terminal, entregar un objeto o recompensa y activar una misión.
- Registrar conversaciones completadas, personajes con quienes se habló y nodos visitados durante la partida actual, para consultarlos desde otras condiciones.

El sistema **no** crea automáticamente una misión ni un objeto nuevo. Las referencias deben apuntar a los catálogos de misiones, ítems o recompensas existentes; si no se configura una referencia de recompensa o misión, no se entrega ni se activa.

¿Cómo se usa?

1. Crear o elegir un catálogo

1. En el panel **Project**, elige una carpeta para los assets de diálogo. El catálogo de ejemplo está en `Assets/_SocialProyectoCenigraf/Asset/Dialogue/DialogueCatalog_Example.asset`.
2. Para crear uno, usa **Create >Cenigraf >Dialogue Catalog**.
3. Abre el asset y añade una entrada en `Dialogues`.
4. Asigna a la entrada un `id` único. Ese ID se escribe también en el componente del NPC.

Un mismo catálogo puede contener conversaciones para distintos personajes. Mantener cada conversación como una entrada con nodos conectados evita que el catálogo crezca con una entrada por cada frase o respuesta.

2. Construir la conversación

Para una conversación ramificada, configura estos campos de la entrada:

Campo	Uso
`id`	Identificador único de la conversación; por ejemplo, `Sala_ABC_Bienvenida`.
`speakerCharacterId`	ID estable del personaje. Al completar la conversación se registra para la condición `TalkedToCharacter`. Si se deja vacío, ese registro no se asocia a un personaje.
`startNodeId`	ID del primer nodo que se presenta cuando no aplica una ruta de inicio condicional.
`conditionalStartRoutes`	Rutas evaluadas al abrir. Se usa la primera condición cumplida; si ninguna coincide, se usa `startNodeId`.
`nodes`	Todas las líneas, respuestas y rutas de la conversación. Cada nodo necesita un `id` único dentro de esa conversación.
`missionCatalog` y `missionId`	Misión que se activa al completar cualquier ruta de la conversación. Déjalos vacíos si no inicia una misión.
`itemCatalog`, `completionItemId`, `completionItemQuantity`	Ítem y cantidad que se agregan al inventario al completar cualquier ruta. Déjalos vacíos si no entrega un ítem.
`completionReward`	Recompensa general al completar la conversación.

En cada nodo:

Campo	Uso

`id`	Nombre de referencia de esta línea, por ejemplo `Line1`, `PreguntarTema` o `Despedida`.

`speaker`	Nombre que se muestra en pantalla.

`text`	Texto de la línea. Admite etiquetas de color TMP y tokens de nombre del jugador.

`nextNodeId`	Siguiente nodo cuando la línea no tiene una opción visible. Usa `*END*` para terminar.

`options`	Respuestas visibles y sus destinos. Cada respuesta normal aparece como botón.

`conditionalRoutes`	Destinos alternativos que se evalúan en orden al avanzar. Si ninguna condición se cumple, se usa el destino por defecto.

`completionItemCatalog`, `completionItemId`, `completionItemQuantity`	Ítem que se entrega solo si esta línea terminal termina la conversación.

`completionReward`	Recompensa específica de esta línea terminal; tiene prioridad sobre la recompensa general.

Ejemplo conceptual de una sola entrada:

```
DialogueSequence
id: Sala_ABC_Bienvenida
startNodeId: Line1
nodes:
Line1
speaker: Recepcionista de Sala ABC
text: ¿Deseas tener más información del ABC SENA?
options:
Si -> Line2
No -> Despedida
Line2
text: La Sala ABC es un lugar donde...
nextNodeId: PreguntarTema
PreguntarTema
text: Dime un tema que te interese.
options:
Misión y visión -> MisionYVision
Actividades del aprendiz -> ActividadesAprendiz
Despedida
text: ąGracias por tu interés!
nextNodeId: *END*
```

Este ejemplo muestra la estructura, no es sintaxis para pegar en el Inspector. Los nodos y opciones se crean en los arreglos correspondientes del asset.

3. Configurar respuestas y destinos

Cada opción contiene:

- `key`: identificador interno. Puede usarse como texto visible si no se escribe `text`; las claves CamelCase se separan en palabras.
- `text`: etiqueta exacta del botón. Úsala cuando necesites acentos, espacios o una frase distinta de la clave.
- `nextNodeId`: ID del nodo que se abrirá al elegir la opción, o `*END*` para cerrar.

Una opción cuya clave sea `Default` **no aparece como botón**. Su destino se usa como avance automático al pulsar continuar. También puedes dejar `Default` fuera de `options` y usar `nextNodeId` del nodo.

La primera condición verdadera de `conditionalRoutes` sustituye el destino por defecto. Si ninguna condición coincide, se usa `nextNodeId` o la opción `Default`. Las rutas se revisan en el orden en que aparecen en el arreglo; pon primero la condición más específica cuando varias puedan cumplirse.

Tipos de condición disponibles:

Condición	Qué comprueba	Valor de `conditionId`
`ItemOwned`	El jugador posee el ítem.	ID del ítem en el catálogo de objetos.
`MissionActive`	La misión está activa.	ID de misión.
`MissionCompleted`	La misión está completada.	ID de misión.
`TalkedToCharacter`	Se completó una conversación asociada al personaje.	`speakerCharacterId` registrado.
`DialogueCompleted`	Se completó la conversación.	ID de la conversación, no el ID de nodo.
`DialogueNodeVisited`	Se alcanzó un nodo concreto durante esta partida.	`DialogueSequence.NodeVisitKey(dialogueId, nodeId)`, con formato `ID_DIALOGO|ID_NODO`.

4. Usar el nombre del jugador en el texto

Escribe cualquiera de estos tokens en `speaker`, `text` o en el texto de una respuesta:

- `*NAME*`: primera palabra del nombre guardado en la sesión.
- `*LASTNAME*`: las palabras restantes, si las hay.
- `*FULLNAME*`: nombre completo.

No distingue mayúsculas y acepta espacios dentro del token, por ejemplo `* Name *`. Si la sesión no tiene un nombre, `*NAME*` y `*FULLNAME*` muestran `Aprendiz`; `*LASTNAME*` queda vacío.

Para colorear el nombre de un camino, usa una etiqueta TMP en el texto, por ejemplo `<color=#E86B78>Camino del Aprendizaje</color>`. Revisa el modo de presentación de la escena: debe admitir el formato de texto que usas.

5. Conectar la entrada al NPC

1. Selecciona el prefab o GameObject del NPC.
2. Asegúrate de que tenga un `NPCInteractionZone` con un collider configurado como trigger, para detectar al jugador dentro del área.
3. Añade o configura `NPCDialogoOfficialInteraction`.
4. Asigna `Dialogue Catalog` y escribe en `Dialogue Id` el `id` exacto de la entrada creada.
5. Configura el retrato si corresponde. Si el NPC tiene `NPCLayeredSkinController`, el retrato y el color de cabeza actuales se usan automáticamente; el retrato manual queda como alternativa.
6. Entra en Play Mode, acércate al NPC y presiona **E**. En móvil, usa el control de interacción contextual.

El componente comprueba que no haya otro diálogo, ventana de recompensa o bloqueo de entrada activo. Si el catálogo o el ID no son válidos, registra una advertencia en la consola y no abre la conversación.

6. Elegir el modo de presentación

`DialoguePresenter.TryOpen` decide qué presentación utilizar:

- Las entradas lineales antiguas se muestran con `DialoguePresenter`.
- Las entradas con nodos se muestran con `DialogoOfficial` cuando el modo es `OfficialPopup`.
- Si el modo es `LegacyCinematic`, los nodos también se muestran con `DialoguePresenter`.

Actualmente, `DebugDialogueSceneMode` selecciona `LegacyCinematic` para la escena cuyo nombre es exactamente `Templo_De_Los_4_Caminos`, y `OfficialPopup` para las demás escenas. Así la presentación se cambia por escena mediante el estado Debug sin duplicar los datos de conversación. Este comportamiento depende del nombre de la escena.

7. Controles durante una conversación

DialogoOfficial / ventana emergente:

- **E** o **Enter**: completar el texto visible y avanzar; si hay respuestas, confirmar la seleccionada.
- **Flechas arriba/abajo**: cambiar la respuesta seleccionada.
- **Esc**: cerrar.
- En móvil, los botones y el control de avance permiten interactuar con la conversación.

DialoguePresenter / presentación cinematográfica:

- **E** o **Enter**: completar la animación del texto y después avanzar o confirmar.
- **Flechas arriba/abajo**: cambiar respuesta seleccionada.
- **Esc**: cerrar.
- El texto de las respuestas aparece después de terminar la animación de la línea.

El movimiento del jugador se pausa mientras el diálogo está abierto y se restaura al cerrarlo. Los NPC y portales también evitan iniciar interacciones superpuestas.

8. Recompensas y seguimiento de progreso

Al terminar, `DialogueCompletion` registra el ID como completado. También registra `speakerCharacterId`, si se configuró. El registro de progreso de `DialogueProgress` vive en memoria durante la partida y se limpia al iniciar una nueva sesión de Play; no es persistencia entre sesiones.

La recompensa se aplica cuando el jugador termina normalmente una conversación con `*END*`. Cerrar con **Esc** no completa las rutas ni entrega sus recompensas. Las opciones que llevan a otro nodo no completan la conversación.

- La recompensa general y el ítem general se aplican a cualquier final de la conversación.
- Los campos de recompensa e ítem en un nodo terminal se aplican al llegar a ese final; la recompensa del nodo reemplaza la general cuando ambas existen.
- La misión general se activa al completar la conversación.

Ejemplo real y archivos de referencia

`Assets/_SocialProyectoCenigraf/Asset/Dialogue/DialogueCatalog_Example.asset` contiene ejemplos vigentes:

- `Sala_ABC_Bienvenida`: conversación ramificada en una sola entrada.
- `Recepcionista_entrega_de_carnet`: uso de `*NAME*` y condición inicial según posesión del carnet.
- `Recepcionista_2_sala_ABC`: rama condicional por posesión de ítem.
- `Estudiante_Desconocido_Portal` y otros diálogos: combinaciones de condiciones y recompensas.

Para revisar el flujo en código:

- `Scripts/Dialogue/DialogueCatalog.cs`: datos, condiciones, tokens y finalización.
- `Scripts/Dialogue/DialoguePresenter.cs`: selección de presentación, entrada y vistas cinematográficas.
- `Scripts/Dialogue/DialogoOfficial.cs`: ventana emergente y opciones.
- `Scripts/World/NPC/NPCDialogoOfficialInteraction.cs`: conexión entre NPC, zona y catálogo.
- `Scripts/Debug/State/DebugDialogueSceneMode.cs`: modo de presentación automático por escena.